Problem A. Quantum Supremacy

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

The era of quantum computers is just around the corner. At least, that is promised by different reports almost every year. Quantum computers are very expensive to build and maintain but can result in orders of magnitude speed-up in some specific tasks. The key parameter of a quantum computer is the number of qubits q it operates with.

Suppose there is a specific cryptographic algorithm that uses a secret binary string of length n. The only known way to crack it using a traditional computer is to try all 2^n possible secret strings and test each in a seconds. For a quantum computer with q qubits there is an algorithm that can simultaneously test up to 2^q secret binary strings in b seconds. These can be arbitrary $k \le 2^q$ binary strings of length n, chosen among all 2^n binary strings of this length.

You want to know when the quantum supremacy starts. Given n, a and b, determine the minimum non-negative integer value of q such that the time it takes to solve the problem (test all secret strings) using a quantum computer only is strictly less than the time it takes to solve the problem using a traditional computer.

Input

The only line of the input contains three integers n, a and b ($1 \le n, a, b \le 10^{18}$), the length of the secret binary string used by cryptography algorithm, the time it takes to test one binary string with a traditional computer and the time it takes to test up to 2^q binary strings using a quantum computer.

Output

If there is no non-negative intever value of q that results in quantum supremacy for this problem, print `-1` in the only line of the output.

Otherwise, print the minimum non-negative integer q such that a quantum computer with q qubits will be faster than a traditional computer.

Examples

standard input	standard output
1024 1 1	1
1024 1 2	2

Problem B. Post-capitalism

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

Long long time ago, in a far away country, n economic agents gathered and built a nation. Initially everyone had 1 dollar. In the following $2020^{2019} - n$ days, 1 dollar of value appeared and went to a randomly chosen person. How was this person chosen? Assume at the beginning of a day, the capitals of people are $s_1, s_2, \ldots, s_n$ dollars. The probability of the i-th person getting the dollar is $\frac{s_i}{s_1+\ldots+s_n}$.

Many years have passed, and now you, a young promising sociologist, are making an inequality research on this old nation. You've managed to ask the poorest person about hir capital. What is **conditional expectation** of the richest person capital?

Formally, let s_i be the number of dollars the i-th agent has after all this time, $\sum_{i=1}^{n} s_i = 2020^{2019}$. Denote by p_i the share of the i-th person, i.e. $p_i = \frac{s_i}{s_1+\ldots+s_n}$. Let $p_{(1)} \leqslant p_{(2)} \leqslant \ldots \leqslant p_{(n)}$ denote the sorted shares. You are given $a = p_{(1)}$. Find conditional expectation $E(p_{(n)}|p_{(1)} = a)$.

For discrete random variables X and Y, and some given value b such that $P(Y = b) > 0$, we define conditional expectation $E(X|Y = b)$ as a weighted average value of X if we only consider events where $Y = b$. Let A be the set of all possible values that random variable X can take. Formally,

$$E(X|Y = b) = \frac{\sum_{a \in A} a \cdot P(X = a, Y = b)}{P(Y = b)}.$$

Input

The only line of the input contains integer n ($2 \leq n \leq 2000$) and real number a ($0 < a \leq \frac{1}{n}$).

It is guaranteed that a is given with no more than nine digits after decimal point, therefore $P(Y = a) > 0$ for any valid a.

Output

Output one real number, the conditional expectation of the maximum share if the minimum share is equal to a. Your answer will be considered correct if its absolute error doesn't exceed 10^{-6}.

Examples

standard input	standard output
2 0.01	0.990000000000
100 0.01	0.010000000000
5 0.198802992	0.201920200333

Problem C. Huawei Frequencies Selection

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	512 megabytes

Huawei prepares to blow up the market with its new super fast mobile device that is too secret to be named here. In order to make something really fast and reliable one needs to pay a lot of attention to details. You are working on a task that will affect how device picks a frequency for data transmission.

There are $n + 1$ different frequencies your new device is able to use. They are numbered from 0 to n with frequency 0 being the most suitable for fast communication, frequency 1 being the most suitable after 0 and so on, n is the least suitable frequency on the list. Unfortunately, you can't always use frequency 0. For each of the next n seconds there is exactly one frequency that will be banned for use because of maintenance. Indices of these frequencies are provided by a sequence of integers $a_1, a_2, \ldots, a_n$, each of them is from 0 to n inclusive.

Your goal is to split these n seconds into k periods, with each period being a non-empty sequence of consecutive seconds, such that each second belongs to exactly one period. Formally, you select integer lengths $l_1, l_2, \ldots, l_k$, such that $\sum_{i=1}^{k} l_i = n$. Then, during the first period there will be a maintenance held at frequencies, $a_1, a_2, \ldots, a_{l_1}$, during the second period there will be a maintenance held at frequencies $a_{l_1+1}, a_{l_1+2}, \ldots, a_{l_1+l_2}$ and so on.

After you fix a split of n seconds into periods, the device will automatically select a frequency for each period as the minimum frequency x that will be available at any second of this period, i.e. minimum integer $x \geq 0$, such that $x \neq a_i$ for any i of this period. Denote the frequencies being selected for these k periods as $f_1, f_2, \ldots, f_k$. Note that these frequencies are selected automatically and the only way you can affect this process is by changing the way periods are chosen.

To care about users safety you will pick one emergency frequency y for all periods. As an emergency frequency it won't be affected by any maintenance, so y can be equal to any a_i, but it can't match any of f_i. One can show there will be always at least one valid y available. Of course, you will pick minimum such y as an emergency frequency.

The goal of this problem is to split n seconds into k periods in such a way that emergency frequency is minimum possible.

Input

The first line of the input contains two integers n and k ($1 \leq k \leq n \leq 1\,000\,000$), the length of the sequence a and the number of periods respectively.

The second line contains the sequence $a_1, a_2, \ldots, a_n$ ($0 \leq a_i \leq n$).

Output

Print one integer, the minimum possible emergency frequency that can be obtained by splitting input sequence into exactly k periods.

Examples

standard input	standard output
2 2 0 2	2
3 1 2 1 1	1
3 2 1 3 0	2

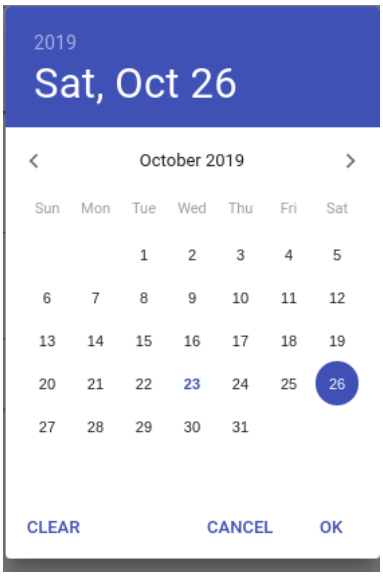
Problem D. JCPC Registration System

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

Joyful Computer Painting Contest (JCPC) is the world biggest painting competition and is extremely popular in Byteland. To make it easier for newbies to join the community, Jeff has created a new registration system.

Krans is responsible for registering participants of JCPC for regional contest in Default City of Byteland and he finds it inconvenient to enter the dates of birth using the interface provided. In this problem you will help him automate this process by implementing the part of the script that enters dates from raw data.

The registration system interface to enter date of birth looks as follows:



At every moment of time there is exactly one currently selected year between 1900 and 2100, and exactly one currently selected month. There are twelve months as usual, with January being the first month and December being the last month. There can be a selection at some day of current month of current year, or there can be no selection of day at all.

There are three types of actions you can make: change selected year, change selected month and select some day.

To change the selected year you can click on it and scroll down to increase selected year, or scroll up to decrease year. If the selected year already matches the one you need the script should do nothing. It is not possible to scroll up to values below 1900 or scroll down to values above 2100. Every time you change the current year, the current month remains the same but the selection of day (if it was set) disappears.

To change the selected month you can click on the button placed to the left of the month's name to decrease it (go into direction of January) or click on the button placed on the right of the month's name to increase it (go into direction of December). Clicking left if the current month is January or clicking right if the current month is December has no effect. Every time you change the current month, the current year remains the same but the selection of day (if it was set) disappears.

To select the day (or to change the selection of day), you must click the corresponding cell in a week-aligned matrix, consisting of 4, 5 or 6 rows and 7 columns. Rows correspond to weeks that have any of their days in this month (weeks can be complete or incomplete), and columns correspond to days of week. Note that in Jeff's calendar every week starts on Sunday and ends on Saturday.

For example, as you can see on the picture above, if the current month has 31 day and first day of the month is Tuesday, then first day of the month is placed at the third cell of the first row, while last day of the month is at the fifth cell of the fifth row.

You are to help t participants register in the system. For each of them you are given the currently selected date and hir date of birth. Your task is to check whether this date is correct, i.e. this day actually existed, and, if so, print the sequence of actions that changes the current day to the given date of birth.

Keep in mind that some years are leap, i.e. contain an additional 29-th day in February. The year is leap if it is divisible by 400, or is divisible by 4 and not divisible by 100. For example, years 1984 and 2000 are leap, while years 1990 and 2100 are not.

Input

The first line of the input contains one integer t ($1 \le t \le 100\,000$) — the number of users to process. Then follow t user's descriptions.

The first line of a user description contains three integers d_c, m_c and y_c ($1 \le d_c \le 31$, $1 \le m_c \le 12$, $1900 \le y_c \le 2100$ — current value of the day, month and year selected in the field. It is guaranteed that this field corresponds to a correct date, i.e. such day exists. The second line of the description contains three integers d_n, m_n and y_n ($1 \le d_n \le 31$, $1 \le m_n \le 12$, $1900 \le y_n \le 2100$ — values of the day, month and year user wants to input in the registration system.

It is guaranteed that the requested date differs from the current date.

Output

For each user print one line in the order user's descriptions appear in the input. If the requested date doesn't exist print "`Unspecified Server Error`". Otherwise, print the command that will select target date in the calendar. Below are the rules on how to construct the command. If the requested date is correct (exists), print the command to enter new date in the format: year changes, month changes, day selection. All non-empty blocks must be separated by a single space.

- If the current year is the same as the requested year, the year change block must be empty. Otherwise, print `u:`n to scroll up (decrease) the current year by n or `d:`n to scroll down (increase) it by n.
- If the current month is the same as the requested month, the month change block must be empty. Otherwise, print `l:`n to click left button n times or `r:`n to click right button n times. You are not allowed to make clicks that have no effect, i.e. click left on January or click right on December.
- Since the target date is always different from the initial date, the day selection part is always non-empty. Print `[`r`][c]`, where r and c are row and column (1-based) of the new date in a week-aligned table of month.

Note, that these rules guarantee that for any existing new date there is exactly one valid sequence of actions that changes the current date to the requested date.

Example

standard input	standard output
4 4 4 2019 30 6 2020 26 10 2019 1 1 2019 26 10 2019 20 10 2018 26 10 2019 29 2 2019	d:1 r:2 [5][3] l:9 [1][3] u:1 [3][7] Unspecified Server Error

Problem E. Raiffeisenbank Logistics

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	512 megabytes

One of important business tasks for Raiffeisenbank is to improve the process of delivering cards, valuables, papers and so forth between banks, clients, pick-up points and other locations. You work in a pilot project that plans to revolutionize the world of delivery by using air drones and optimal routing algorithms.

You are now performing the very first tests at onsite training ground that consists of n landing spots. There is a drone at spot 1 and it needs to fly to spot n. Drone's remote control can run m routing programs, the i-th program is able to move drone from spot u_i to spot v_i (if the drone is not at spot u_i, the program will do nothing), and it was written using t_i version of your drone control library. As this is one of the first tests of your project you are not sure about compatibility of different versions of software, so each routing program you run (except for the very first) must have version of drone control library strictly greater, then the previous one.

After looking at the code of routing programs you found out that you can make little changes there. In particular, you can take any program i and swap its starting and destination spots. So, if the i-th program moves drone from spot u_i to spot v_i, after the changes it will move from from spot v_i to spot u_i. Library version t_i will remain unchanged.

Now you wonder, what is the minimum number of programs you need to change, so there will exist a sequence of programs $p_1, p_2, \ldots, p_k$ that being run in this order moves drone from spot 1 to spot n, and $t_{p_1} < t_{p_2} < t_{p_3} < \ldots < t_{p_k}$.

Input

The first line of the input contains a single integer t ($1 \le t \le 1000$) — the number of test cases. Then follow t tests descriptions.

Each test description starts with two integers n and m ($2 \le n \le 500\,000$, $1 \le m \le 500\,000$), the number of landing spots at the training ground and the number of routing programs respectively.

Then follow m lines, the i-th of them describes the i-th program with three integers u_i, v_i and t_i ($1 \le u_i, v_i \le n$, $1 \le t_i \le 10^9$), meaning this program moves drone from spot u_i to spot v_i and uses t_i-th version of drone control library.

There can be multiple programs with the same version of software. There can be multiple programs connecting the same pair of spots (in each direction). There can be programs that move drone from some spot to itself.

It is guaranteed that sum of n over all tests won't exceed 500 000. It is guaranteed that sum of m over all tests won't exceed 500 000.

Output

If there is no way to change programs and move drone from spot 1 to spot n obeying condition that every next program you run must have greater version of drone control library, print `-1` in the only line of the output.

Otherwise, print the minimum number of programs you need to change to achieve the goal.

Examples

standard input	standard output
1 4 3 2 1 1 2 3 2 4 3 3	2
2 4 3 2 1 1 2 3 2 4 3 2 8 9 1 2 5 2 3 10 4 3 15 4 5 20 5 8 25 1 6 2 6 5 30 7 6 3 8 7 4	-1 1

Problem F. Toll Road

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	512 megabytes

A new shiny triple-A class toll highway from Moscow to Saint-Petersburg is about to open soon. Of course, this highway is bi-directional, but for the purpose of this problem we only consider direction from Moscow to Saint-Petersburg as the one all participants of this competition would like to travel in late November.

The road will be of length L (in some specific units of distance who wished to remain anonymous) and will have n special points. All special points are located at some distinct integer coordinates along the road, with one of them being located at position 0 and one at position L. Each special point is either an entrance or an exit. Of course, special point at position 0 is an entrance and special point at position L is an exit.

Each car that uses the highway will be equipped with a special device called transponder. The owner of the road will place toll gates at some positions along the road to communicate with transponders of cars passing by. There are two types of gates that can be built along the road.

1. Entrance or exit gates. They can only be built at positions that contain an entrance or an exit. Such gate costs a burles to build and communicates with transponders of all cars that use corresponding entrance or exit.
2. Road gates. They can be built only at half-integer positions, i.e. points $k + 0.5$ for all k from 0 to $L - 1$. Such gate costs b burles and communicates with transponders of all cars passing by along the highway.

In order to properly charge all cars that use the highway, one needs to be able to identify the exact entrance position and the exact exit position for each of these cars. Formally, you need to build a set of gates to satisfy the following requirements.

1. Each car that enters the road at some entrance, goes forward and then takes some exit should pass near at least one of the gates, so we can identify that this car was using the highway.
2. For each car that enters the road at some entrance, goes forward and then takes some exit we should be able to exactly tell the position of the entrance and the position of the exit based on the information from communication between this car's transponder and toll gates. Note that due to complicated tariffs we are interested in exact positions of the entrance and the exit, not just the distance travelled along the highway.
3. The total cost of all gates built should be minimum possible.

Input

The first line of the input contains a single integer t ($1 \le t \le 1000$) — the number of tests to process in this input. Then follow t tests' descriptions.

The description of each test starts with four integers n, L, a and b ($2 \le n \le 500\,000$, $1 \le L, a, b \le 10^9$), the number of special points along the highway, the length of the highway, the cost of building one gate at entrance or exit, and the cost of building one gate along the road respectively. Each of the next n lines contains a character c_i and an integer x_i ($0 \le x_i \le L$), separated by a single space. The character c_i denotes the special point type and is either '`E`' (entrance) or '`T`' (exit), while x_i means the position of this point along the road. It is guaranteed that all x's are distinct, and there is an entrance at position 0 and an exit at position L.

It is guaranteed that sum of n over all tests won't exceed 500 000.

Output

For each test print one integer on a separate line, the minimum total cost of building a set of gates that gives a full control of movements of all cars along the highway.

Example

standard input	standard output
4 2 10 1 1 E 0 T 10 2 10 3 2 E 0 T 10 5 40 5 6 E 0 T 40 T 20 E 30 E 10 10 9 5 6 T 5 E 6 T 7 E 8 T 9 E 0 T 1 E 2 T 3 E 4	1 2 15 28

Problem G. Yandex Museum

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	512 megabytes

Yandex is a Russian technology company developing lots of products that make our everyday life easier: web search, e-mail, taxi service, maps, and so on. Many of Yandex products are running on the most modern intelligent devices varying from computers and mobile phones to self-driving cars and smart home assistants. But there were times when computers were less user-friendly, much less efficient and required true technical skills just to run some program. Does any of the names "Commodore 64", "Elektronika MK-90", "ZX Spectrum 128" and "Apple IIe" raise nostalgia feelings in you? For some Yandex employees, these words mean a lot. That's why Yandex has created a special Yandex Museum that is free and open for anybody to visit, and has an exhibition of old computers including all aforementioned legendary devices!

Arkady decided to bring to the museum his old computer which is not functioning any more. This computer has a rather primitive display, which was able to display only four colors: black, red, green and blue. Still, Arkady spent a lot of time with that computer back in the day, especially with the primitive image editing program.

Initially, when user starts working on an image, the drawing area is filled with red color. The favorite drawing tool of Arkady is a triangle tool, which works as follows: user chooses three distinct integer points A, B and C defining a non-degenerate triangle (i.e. of positive area), and then editor draws a triangle $\triangle ABC$ with the given points as its vertices. As you remember, the screen is able to display only few colors, so the coloring logic is following.

- All points belonging to the sides of the triangle (to at least one of the segments AB, BC and AC) are painted black. Once painted black, point never changes its color and remains black till the very end.
- Each non-black point in the triangle interior changes its color to the next one in order of "RGB", i.e. every point that is currently red is colored green, every green point is colored blue and every blue point becomes red.
- Points that are strictly outside of the triangle do not change their colors.

Arkady used to enjoy drawing triangles, but he never liked blue and green colors, so he was especially happy when after some drawing steps picture consisted of black and red colors only (by the way, same holds for Yandex logo!). He called such pictures *nice*.

While packing the computer to bring it to the museum, Arkady found an old piece of paper containing n sextuplets of integers. He supposed that these integers are the coordinates of vertices of n triangles he has once drawn during his younger days. Indeed, he noticed that all triangles are non-degenerate, but he was not able to figure out by simply looking at these numbers, whether the resulting picture is nice or not. Help him find the answer for that question!

Input

The first line of the input contains an integer n ($1 \le n \le 100\,000$), the number of triangles.

Each of the following n lines contains six integers which are the coordinates of the triangle. Each triangle is non-degenerate, and each coordinate is between 0 and 10^4, inclusive.

Output

If the resulting picture is nice, print a single word "`nice`".

Otherwise, on the first line print "`not nice`", and on the second line print two (not necessarily integer) numbers, which are coordinates of any point having blue or green color. The numbers should be printed

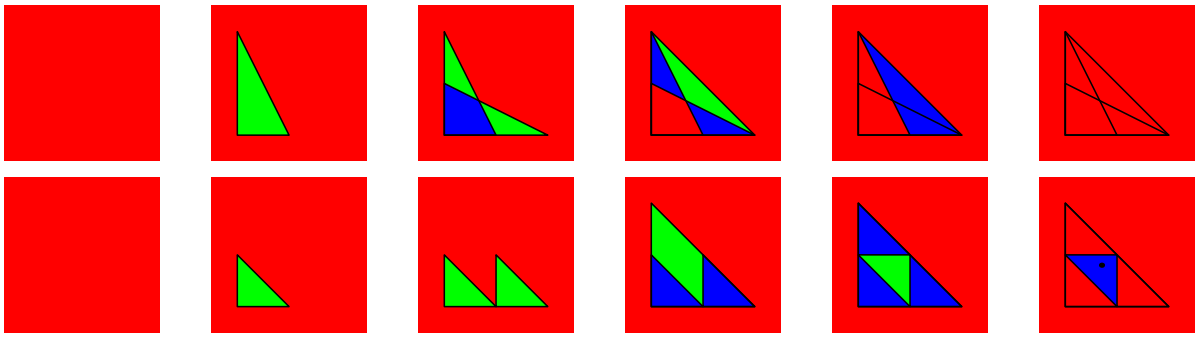
as integers or decimal fractions with at least 0 and **at most** 9 digits after decimal point. Coordinates should be between 0 and 10^4, inclusive. Note that usage of exponential format **is not allowed**. Please, also note that your answer will be verified with no absolute or relative tolerance.

Examples

standard input	standard output
5 0 0 0 2 1 0 0 0 2 0 0 1 0 0 0 2 2 0 2 0 0 1 0 2 0 2 1 0 2 0	nice
5 0 0 1 0 0 1 1 0 2 0 1 1 0 0 0 2 2 0 0 1 0 2 1 1 0 0 2 0 0 2	not nice 0.7 0.8

Note

Illustrations for sample cases are given below:



Problem H. Rebellious Sequences

Input file:	`standard input`
Output file:	`standard output`
Time limit:	4 seconds
Memory limit:	512 megabytes

A bracket sequence is a sequence of opening and closing brackets. A bracket sequence is called *balanced* if and only if two conditions hold:

- it contains equal number of opening and closing brackets;
- for each prefix, the number of closing brackets is not greater than the number of opening brackets.

A bracket sequence is called *rebellious* if it contains equal number of opening and closing brackets and is **not** balanced.

You are given a balanced bracket sequence of n brackets and q queries to it. Each query asks you to swap two brackets. It is guaranteed that after each query the sequence stays balanced.

After each query you should find whether it is possible to split the sequence into several disjoint rebellious subsequences.

Formally, let the original sequence be $s_1 s_2 \dots s_n$. You have to check if there exists a number $k > 1$ (the number of subsequences) and k non-empty sequences of indices $(i_{1,1} < \dots < i_{1,l_1}), \dots, (i_{k,1} < \dots < i_{k,l_k})$ such that for each $1 \le j \le k$ the bracket sequence $s_{i_{j,1}} s_{i_{j,2}} \dots s_{i_{j,l_j}}$ is rebellious and every integer from 1 to n appears among those sequences exactly once.

Input

The first line of the input contains two numbers n and q ($1 \le n, q \le 300\,000$), the length of the sequence and the number of queries respectively. In the second line, there is a string of length n consisting of characters '`(`' and '`)`'.

Then follow q lines that describe the queries. Each of them contains two integers i and j ($1 \le i < j \le n$), denoting that you should swap the brackets on i-th and j-th position.

It is guaranteed that the sequence is balanced initially and remains balanced after each query.

Output

After each query print "`Yes`", if it is possible to split the sequence into several disjoint rebellious subsequences, and "`No`" otherwise.

Example

standard input	standard output
8 4 (()()()) 3 4 5 6 2 7 6 7	No No Yes No

Problem I. Minimax Limit

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	512 megabytes

Min and Max have a continuous piecewise linear function $f(x)$. The function $f(x)$ is defined on the segment $[0, n]$ as follows:

- if $x \in [0, n]$ is an integer, then $f(x) = a_x$;
- otherwise, if $k = \lfloor x \rfloor$, then $f(x) = f(k) + (x - k)(f(k + 1) - f(k))$.

Min and Max are playing a game. Before the game starts they choose two positive real parameters A and ε. There is a counter T initially equal to A. Before the game starts, Max chooses an arbitrary real number $x \in [0, n]$. Players then proceed to take turns starting from Min. During a player's turn:

- if $T \le 0$, the game is over;
- the player can change x to any real number $x' \in [0, n]$ such that $|x - x'| \le T$;
- T is decreased by ε.

When the game is over, the result of the game is equal to $f(x)$. Min wants to minimize the result, while Max wants to maximize it.

Let $R(A, \varepsilon)$ be the result of the game with parameters A and ε if both players play optimally. Compute $\lim\limits_{\varepsilon \to 0} R(A, \varepsilon)$. It is guaranteed that this limit exists.

Input

The first line contains two integers n and A ($1 \le A \le n \le 100\,000$).

The second line contains $n + 1$ integers $a_0, \ldots, a_n$ ($-10^6 \le a_i \le 10^6$).

Output

Print a single real number $\lim\limits_{\varepsilon \to 0} R(A, \varepsilon)$. Your answer will be accepted if its absolute or relative error does not exceed 10^{-4}.

Examples

standard input	standard output
1 1 0 1	0.5000000000
2 1 0 2 1	1.4285714283

Problem J. You Are Given a Tree

Input file:	`standard input`
Output file:	`standard output`
Time limit:	5 seconds
Memory limit:	512 megabytes

Vlad doesn't like to read. Grisha likes formal statements. Vlad came up with the task. Grisha wrote a statement. As there was no way you can find a story here, they would like you to recall that a tree of size n is a connected undirected graph with n vertices and $n-1$ edges. You are given a tree of size n. We denote as V ($|V| = n$) the set of its vertices. Vertices are numbered with consecutive integers from 1 to n.

For any $S \subset V$, we define its *beauty* as a size of the smallest set of vertices T ($T \subset V$) such that for any two vertices $u, v \in S$ all vertices on the unique simple path between u and v belong to T.

For any l and r ($1 \le l \le r \le n$) we define $S(l, r)$ as a set of vertices $\{l, l+1, l+2, ...r-1, r\}$.

Find the sum of the beauties of $S(l, r)$ for all $1 \le l \le r \le n$.

Input

The first line of the input contains a single integer n ($2 \le n \le 300\,000$), the number of vertices in the tree.

Next $n-1$ lines contain the description of the tree. The i-th of these lines (lines are numbered from 1) contains a single integer p_i ($1 \le p_i \le i$) describing an edge between vertices p_i and $i+1$.

Output

Output a single integer — the sum of the beauties of all $S(l, r)$.

Examples

standard input	standard output
2 1	4
3 1 1	11

Note

In the second sample, there are six sets.

1. $S = \{1\}, T = \{1\}$;
2. $S = \{2\}, T = \{2\}$;
3. $S = \{3\}, T = \{3\}$;
4. $S = \{1, 2\}, T = \{1, 2\}$;
5. $S = \{2, 3\}, T = \{1, 2, 3\}$;
6. $S = \{1, 2, 3\}, T = \{1, 2, 3\}$.

Problem K. Dynamic Traffic with MegaFon

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	512 megabytes

You are a young and talented engineer working at MegaFon — one the largest telecommunication companies in Russia. Your current project aims to improve mobile internet experience by dynamically allocating outgoing traffic limits for each user based on the current consumption.

As a subtask of this project you are given a sequence of integers $a_1, a_2, \ldots, a_n$ that describes communication between user's phone and your transmitter for some period of n consecutive seconds. Value a_i equals to the difference between traffic sent and traffic received during second i, thus, this values can be positive, negative, or equal to zero.

For any sequence of traffic differences $b_1, b_2, \ldots, b_k$ you define its *weak estimation* as the sum of maximums of all pairs of adjacent elements, i.e. $f(b_1, b_2, \ldots, b_k) = \sum_{i=1}^{k-1} \max(b_i, b_{i+1})$. Note that weak estimation of empty sequence equals to 0.

For traffic scheduling you plan to use *strong estimation* of sequence a that you define as the maximum value of weak estimation among all **subsequences** of a. Recall that subsequence is any sequence that you can obtain by removing some elements of the original sequence. Empty subsequence and subsequence equal to the given sequence are also allowed. The task here is to develop a fast algorithm that computes strong estimation of any given sequence.

Input

The first line of the input contains a single integer n ($1 \le n \le 500\,000$), length of sequence a.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($-10^9 \le a_i \le 10^9$).

Output

Output one integer — the value of strong estimation of sequence a.

Examples

standard input	standard output
4 5 -3 -2 1	6
4 -1 -1 -1 -1	0